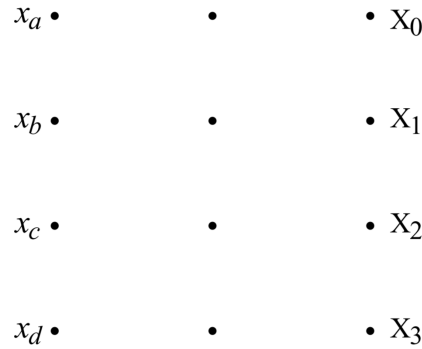


UNIVERSITY OF ILLINOIS AT URBANA-CHAMPAIGN
 Department of Electrical and Computer Engineering
 ECE 310 DIGITAL SIGNAL PROCESSING
Homework 10

Prof. Kamalabadi, Katselis, Liang

Due: 5 pm, April 8, 2022

1. Complete the following signal flow diagram (butterfly structure) of a 4-pt, radix-2, decimation-in-time FFT algorithm. Specify all the connection weights and determine the indexes (a , b , c , and d) of the input signal sequence.



Solution

Recall that a radix-2 decimation-in-time FFT first splits the signal into its even and odd samples. We want to calculate the DFT of the even samples, x_0 and x_2 , and the odd samples, x_1 and x_3 . Therefore,

$$a = 0, \quad b = 2, \quad c = 1, \quad d = 3.$$

We can find the butterfly diagram by using the DFT definition:

$$\begin{aligned} X[k] &= \sum_{n=0}^3 x[n]e^{-j\frac{2\pi}{4}nk} = x_0 + x_1e^{-j\frac{2\pi}{4}k} + x_2e^{-j\frac{2\pi}{4}2k} + x_3e^{-j\frac{2\pi}{4}3k} \\ &= x_0 + x_1(-j)^k + x_2(-1)^k + x_3(j)^k. \end{aligned}$$

We can evaluate this for each $0 \leq k \leq 3$ to find

$$\begin{aligned} X_0 &= x_0 + x_1 + x_2 + x_3, \\ X_1 &= x_0 - jx_1 - x_2 + jx_3, \\ X_2 &= x_0 - x_1 + x_2 - x_3, \\ X_3 &= x_0 + jx_1 - x_2 - jx_3. \end{aligned} \tag{1}$$

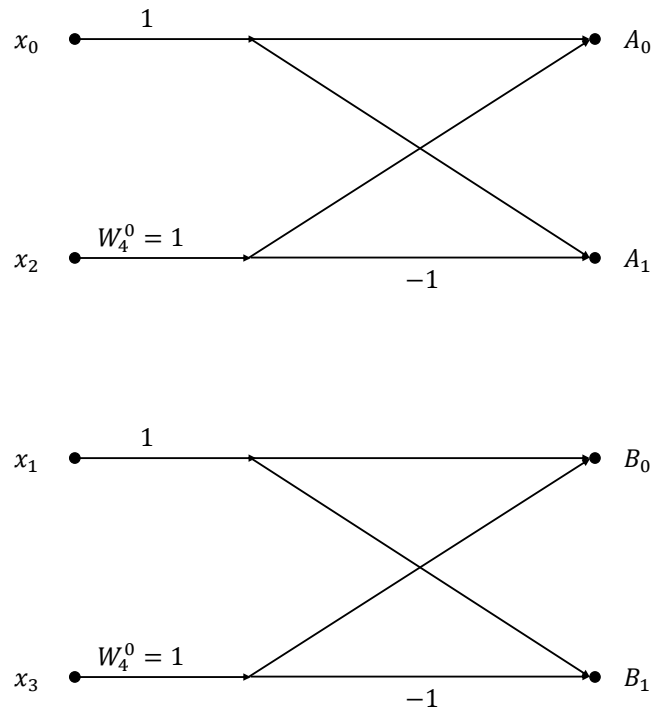
This can be rearranged to show (given $W_4^1 = e^{-j\frac{2\pi}{4}} = -j$)

$$\begin{aligned} X_0 &= (x_0 + x_2) + (x_1 + x_3), \\ X_1 &= (x_0 - x_2) + (x_1 - x_3)W_4^1, \\ X_2 &= (x_0 + x_2) - (x_1 + x_3), \\ X_3 &= (x_0 - x_2) - (x_1 - x_3)W_4^1. \end{aligned}$$

We can define two sub-sequences, $\{A_0, A_1\} = \{(x_0 + x_2), (x_0 - x_2)\}$ and $\{B_0, B_1\} = \{(x_1 + x_3), (x_1 - x_3)\}$ such that

$$\begin{aligned} X_0 &= A_0 + B_0, \\ X_1 &= A_1 + B_1 W_4^1, \\ X_2 &= A_0 - B_0, \\ X_3 &= A_1 - B_1 W_4^1. \end{aligned} \tag{2}$$

This is how we can empirically derive the two-step process for completing the 4-point FFT. We notice the symmetry in the formation of this DFT in that we can construct A and B as differences and sums of our input signal first, then add and subtract these for the final result. We can summarize this in a butterfly diagram as follows: first, we can use two two-point DFT's to construct A and B :

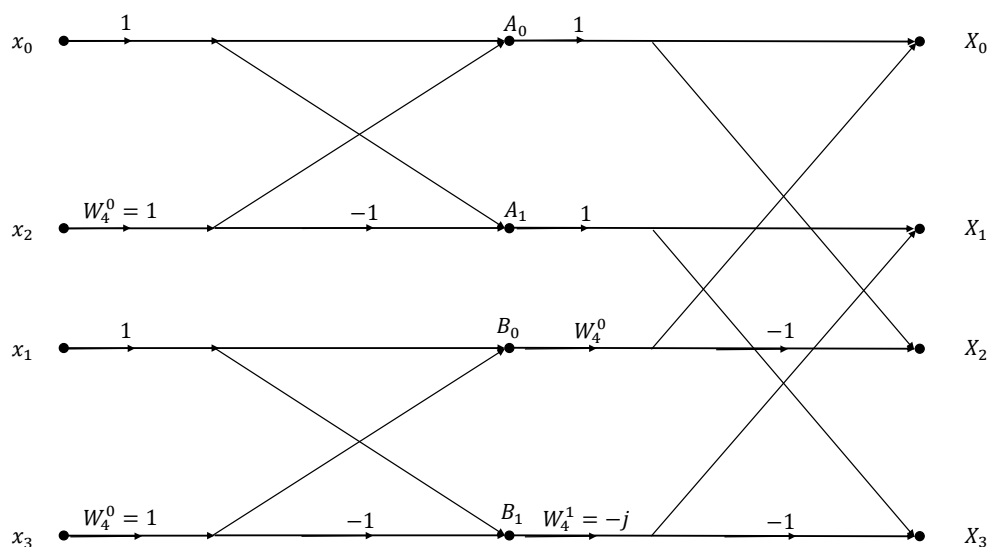


These two sub-butterflies can be read left to right along a particular path. Where arrows meet, their quantities are summed; when an arrow has a number next to it, that is a multiplier; when there is nothing next to an arrow, it has a multiplier of 1. In this case, we can read from the first butterfly that

$$A_0 = x_0 + W_4^0 x_2 = x_0 + x_2, \quad A_1 = x_0 + (W_4^0 x_2)(-1) = x_0 - x_2,$$

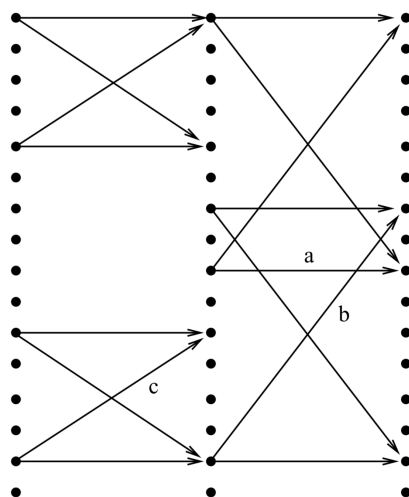
confirming our analytic definition from above. Now that we have calculated A and B , we can use (2) to calculate each X component by linking the two butterfly diagrams in another stage, shown in the figure below.

Notice that this can be derived with a set of general rules rather than by inspection from the DFT definition. The straight branches will be 1 in the upper half of each stage, the diagonal branches coming from the top half all take the value 1, the m^{th} diagonal branch going up takes the value W_N^{m-1} , and the m^{th} straight branch in the bottom half takes the value $-W_N^{m-1}$.



Notice that the computation of A and B each requires 2 additions for a total of 4; then, the computation of X requires 2 multiplications and 4 additions, for a total of 8 additions and 2 multiplications. Doing the direct DFT method from (1) requires 12 additions and 4 multiplications. Clearly, for a system who can do a certain number of multiplications/additions every unit of time (like most modern computers), this special way of computing the DFT is *faster* than the brute-force method. That is why this method, graphically demonstrated with the butterfly diagram, is called the *fast fourier transform*.

- The diagram below represents a part of the computation in a 16-point decimation-in-time radix-2 FFT. Indicate the values of the three requested branch weights a, b and c.



Solution

This can be done by first recalling the rules of the butterfly diagram weights:

- Branches going across in the top half are always 1.
- Branches going diagonally down from the top half are always 1.

- (c) Branches going diagonally up from the bottom half are W_N^m , where N is the current DFT size, and m is the index of the output node in the upper half the branch is heading towards.
- (d) Branches going straight across in the bottom half are $-W_N^m$, where W_N^m is the value of the corresponding diagonal branch leaving the node.

We can apply these rules to each requested value. Note that this figure corresponds to two eight-point DFT's from the left column to the middle and one 16-point DFT from the middle to the right. We can split the left hand side down its middle to reveal the first and second eight-point DFT's. See the figure below.

Upon doing this, we can identify that c is the weight of the branch in the **bottom half** of the lower eight-point DFT, and it points to the **third** output ($m = 3 - 1 = 2$, as we count starting from zero) of the eight-point DFT. As such, we can conclude from rule (c) that

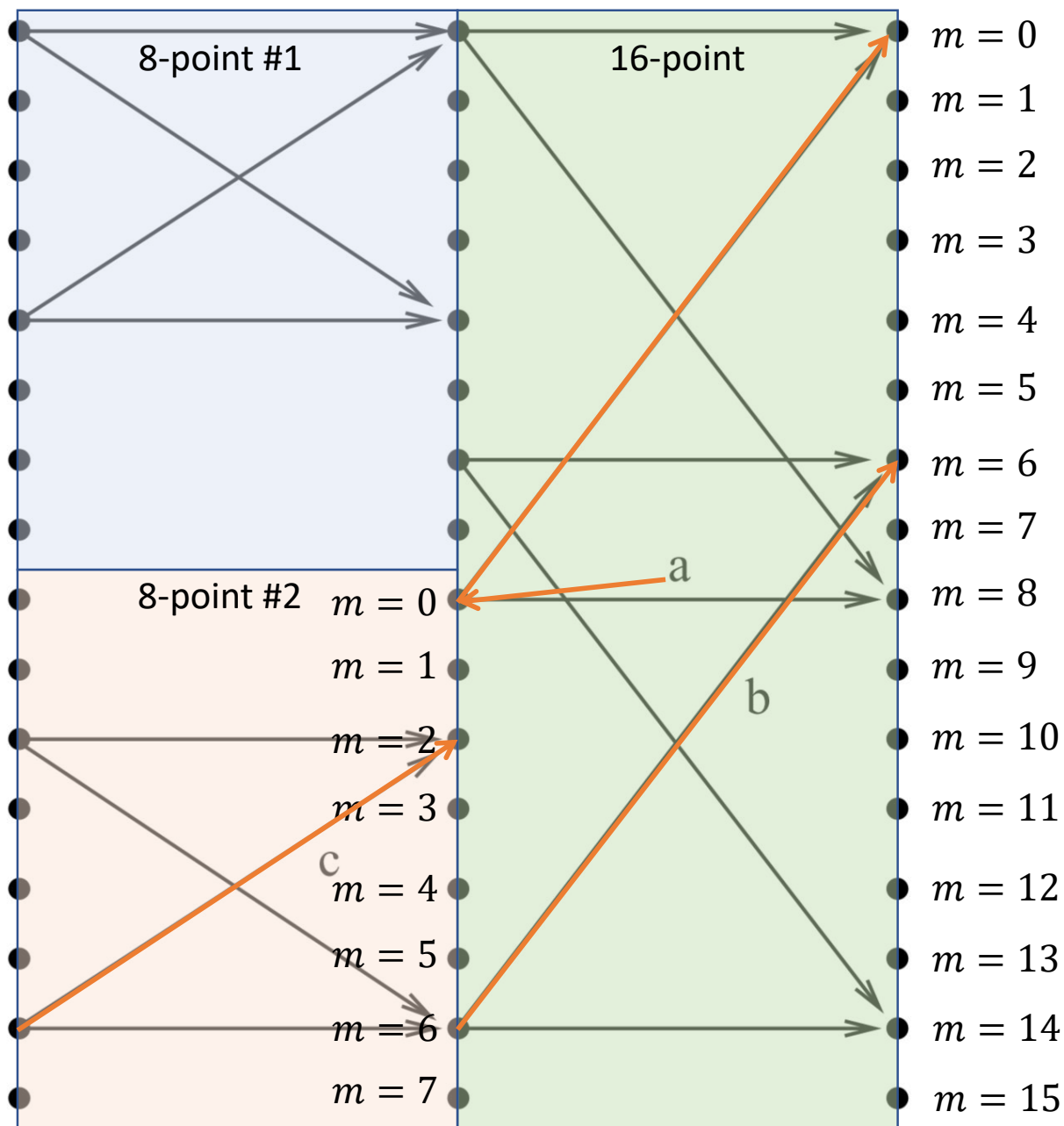
$$c = W_8^{3-1} = W_8^2 = e^{-j\frac{2\pi}{8}2} = e^{-j\frac{\pi}{2}} = -j.$$

We can identify a as a horizontal branch in the bottom half of the 16-point DFT, whose corresponding diagonal points to the zeroth output (the arrow pointing to the top right, $m = 0$). As such, we can conclude

$$a = -W_{16}^0 = -1.$$

Finally, since b represents the weight of the seventh diagonal branch in the bottom half of the 16-point DFT (the diagonal branch corresponding to output $m = 7 - 1 = 6$), we can conclude from (c) that

$$b = W_{16}^{7-1} = W_{16}^6 = W_8^3 = e^{-j\frac{2\pi}{8}3} = e^{-j\frac{3\pi}{4}} = \frac{-1-j}{\sqrt{2}}.$$



3. Consider the two finite-length sequences:

$$x = \{1, 2, 3, 4, 5\}, \text{ and } h = \{1, 1\}$$

- Compute the linear convolution $x * h$.
- Compute the circular convolution $x \otimes_5 h$.
- What is the smallest value of N so that N -point circular convolution is equal to the linear convolution?

Solution

(a) We can compute the linear convolution by noting that $h[n] = \delta[n] + \delta[n-1]$, so

$$x[n] * h[n] = x[n] * (\delta[n] + \delta[n-1]) = x[n] * \delta[n] + x[n] * \delta[n-1] = x[n] + x[n-1].$$

This formula can be evaluated for each n given the definition of $x[n]$, which results in

$$x[n] * h[n] = \{1, 2, 3, 4, 5, 0\} + \{0, 1, 2, 3, 4, 5\} = \{1, 3, 5, 7, 9, 5\}.$$

This can be solved more robustly given the definition of convolution, $x[n] * h[n] = \sum_{l=-\infty}^{\infty} x[l]h[n-l] = \sum_{l=-\infty}^{\infty} x[n-l]h[l]$.

(b) We can compute the circular convolution via its definition, $y[n] = (x \oplus_5 h)[n] = \sum_{l=0}^{N-1} x[m]h_{zp}[\langle n-l \rangle_N]$, which can be done only after zero-padding $h[n]$ to length $N = 5$ to match that of $x[n]$:

$$\{h_{zp}[n]\}_{n=0}^4 = \{h[0], h[1], 0, 0, 0\} = \{1, 1, 0, 0, 0\}.$$

We can do the convolution index-by-index as follows:

$$\begin{aligned} y[0] &= \sum_{l=0}^4 x[l]h_{zp}[\langle 0-l \rangle_5] = x[0]h_{zp}[0] + x[1]h_{zp}[4] + x[2]h_{zp}[3] + x[3]h_{zp}[2] + x[4]h_{zp}[1] = 6, \\ y[1] &= \sum_{l=0}^4 x[l]h_{zp}[\langle 1-l \rangle_5] = x[0]h_{zp}[1] + x[1]h_{zp}[0] + x[2]h_{zp}[4] + x[3]h_{zp}[3] + x[4]h_{zp}[2] = 3, \\ y[2] &= \sum_{l=0}^4 x[l]h_{zp}[\langle 2-l \rangle_5] = x[0]h_{zp}[2] + x[1]h_{zp}[1] + x[2]h_{zp}[0] + x[3]h_{zp}[4] + x[4]h_{zp}[3] = 5, \\ y[3] &= \sum_{l=0}^4 x[l]h_{zp}[\langle 3-l \rangle_5] = x[0]h_{zp}[3] + x[1]h_{zp}[2] + x[2]h_{zp}[1] + x[3]h_{zp}[0] + x[4]h_{zp}[4] = 7, \\ y[4] &= \sum_{l=0}^4 x[l]h_{zp}[\langle 4-l \rangle_5] = x[0]h_{zp}[4] + x[1]h_{zp}[3] + x[2]h_{zp}[2] + x[3]h_{zp}[1] + x[4]h_{zp}[0] = 9. \end{aligned}$$

As such,

$$\{y[n]\}_{n=0}^4 = \{6, 3, 5, 7, 9\}.$$

This can be done similarly with the tabular method:

l	0	1	2	3	4		
$x[l]$	1	2	3	4	5	n	$y[n]$
$h_{zp}[\langle 0-l \rangle_5]$	1	0	0	0	1	0	$1+5=6$
$h_{zp}[\langle 1-l \rangle_5]$	1	1	0	0	0	1	$1+2=3$
$h_{zp}[\langle 2-l \rangle_5]$	0	1	1	0	0	2	$2+3=5$
$h_{zp}[\langle 3-l \rangle_5]$	0	0	1	1	0	3	$3+4=7$
$h_{zp}[\langle 4-l \rangle_5]$	0	0	0	1	1	4	$4+5=9$

to retrieve the same result.

(c) The lowest value of N such that the circular and linear convolution of two sequences x and h of length $L = 5$ and $M = 2$, respectively, should be $N = 5 + 2 - 1 = 6$. This means we will have to pad x and h each to length 6 to have their convolution equal the linear convolution. Notice that the two are not equal when padded to $N = 5$. When we try $N = 6$,

l	0	1	2	3	4	5		
$x_{zp}[l]$	1	2	3	4	5	0	n	$y[n]$
$h_{zp}[\langle 0-l \rangle_6]$	1	0	0	0	0	1	0	$1+0=1$
$h_{zp}[\langle 1-l \rangle_6]$	1	1	0	0	0	0	1	$1+2=3$
$h_{zp}[\langle 2-l \rangle_6]$	0	1	1	0	0	0	2	$2+3=5$
$h_{zp}[\langle 3-l \rangle_6]$	0	0	1	1	0	0	3	$3+4=7$
$h_{zp}[\langle 4-l \rangle_6]$	0	0	0	1	1	0	4	$4+5=9$
$h_{zp}[\langle 5-l \rangle_6]$	0	0	0	0	1	1	5	$5+0=5$

confirming the fact that $x * h = x_{zp} \circledast_6 h_{zp}$. Convince yourself that this remains true for $N > 6$.

4. Determine z_n , the cyclic convolution of x_n and y_n for the following cases:

- (a) $\{x_n\}_{n=0}^5 = \{1, 2, 3, 4, 5, 6\}$ and $\{y_n\}_{n=0}^5 = \{1, 0, 0, 1, 0, 0\}$.
(b) $\{x_n\}_{n=0}^8 = \{1, 2, 3, 4, 5, 6, 0, 0, 0\}$ and $\{y_n\}_{n=0}^8 = \{1, 0, 0, 1, 0, 0, 0, 0, 0\}$.

Solution

(a) This can be done most efficiently via the tabular method. Note that no zero padding has to occur because this cyclic convolution is between two equal-length sequences:

l	0	1	2	3	4	5		
$x[l]$	1	2	3	4	5	6	n	$z[n]$
$y[\langle 0-l \rangle_6]$	1	0	0	1	0	0	0	$1+4=5$
$y[\langle 1-l \rangle_6]$	0	1	0	0	1	0	1	$2+5=7$
$y[\langle 2-l \rangle_6]$	0	0	1	0	0	1	2	$3+6=9$
$y[\langle 3-l \rangle_6]$	1	0	0	1	0	0	3	$4+1=5$
$y[\langle 4-l \rangle_6]$	0	1	0	0	1	0	4	$5+2=7$
$y[\langle 5-l \rangle_6]$	0	0	1	0	0	1	5	$6+3=9$

As such,

$$\{z[n]\}_{n=0}^5 = \{5, 7, 9, 5, 7, 9\}.$$

(b) This can be done most efficiently via the tabular method. Note that no zero padding has to occur because this cyclic convolution is between two equal-length sequences:

l	0	1	2	3	4	5	6	7	8		
$x[l]$	1	2	3	4	5	6	0	0	0	n	$z[n]$
$y[\langle 0-l \rangle_9]$	1	0	0	0	0	0	1	0	0	0	$1+0=1$
$y[\langle 1-l \rangle_9]$	0	1	0	0	0	0	0	1	0	1	$2+0=2$
$y[\langle 2-l \rangle_9]$	0	0	1	0	0	0	0	0	1	2	$3+0=3$
$y[\langle 3-l \rangle_9]$	1	0	0	1	0	0	0	0	0	3	$4+1=5$
$y[\langle 4-l \rangle_9]$	0	1	0	0	1	0	0	0	0	4	$5+2=7$
$y[\langle 5-l \rangle_9]$	0	0	1	0	0	1	0	0	0	5	$6+3=9$
$y[\langle 6-l \rangle_9]$	0	0	0	1	0	0	1	0	0	6	$0+4=4$
$y[\langle 7-l \rangle_9]$	0	0	0	0	1	0	0	1	0	7	$0+5=5$
$y[\langle 8-l \rangle_9]$	0	0	0	0	0	1	0	0	1	8	$0+6=6$

As such,

$$\{z[n]\}_{n=0}^5 = \{1, 2, 3, 5, 7, 9, 4, 5, 6\}.$$

5. The following linear convolution

$$\{x_n\}_{n=0}^{46} * \{h_n\}_{n=0}^{32}$$

is to be evaluated using the DFT method. Namely,

$$\{x_n\}_{n=0}^{46} * \{h_n\}_{n=0}^{32} = \text{DFT}^{-1}\{\text{DFT}\{x_n\} \cdot \text{DFT}\{h_n\}\}$$

- Determine the minimum number of zeros should be padded to $\{x_n\}$ and $\{h_n\}$, respectively, before the DFTs are applied.
- If the DFTs are to be calculated with a radix-2 FFT algorithm, how many zeros should now be padded to $\{x_n\}$ and $\{h_n\}$, respectively.
- In (a), can the zeros be padded at the beginning (instead of the end) of the sequences? If so, how do you obtain $\{x_n\}_{n=0}^{46} * \{h_n\}_{n=0}^{32}$ from $\text{DFT}^{-1}\{\text{DFT}\{x_n\} \cdot \text{DFT}\{h_n\}\}$?

Solution

(a) Notice that this process is effectively computing the circular convolution of x and h ; by the DFT property that

$$x_{zp,e} \otimes_N h_{zp,e} \leftrightarrow X_{zp,e}[k]H_{zp,e}[k] = \text{DFT}\{x_n\} \cdot \text{DFT}\{h_n\}$$

where $x_{zp,e}[n]$, $h_{zp,e}[n]$ are $x[n]$ and $h[n]$ padded on their tail ends to length $N = L + M - 1$, and $X_{zp,e}[k]$, $H_{zp,e}[k]$ are their DFT's. See (7.152) from the text, and read over section 7.5 for more information. Effectively, the products of the DFT's of x and h is the DFT of the circular convolution of $x_{zp,e}$ and $h_{zp,e}$. We know that if we zero-pad to length $N = L + M - 1$, then this this circular convolution equal to the linear convolution of x and h . If we zero-pad according to that requirement, then the IDFT of the DFT product will be the linear convolution. Using this formula, we must zero pad to length $N = L + M - 1 = 47 + 33 - 1 = 79$, 32 zeros to x and 46 to h .

(b) Recall that radix-2 FFT algorithms require signals to be of length equal to a power of 2. With this requirement and that of above ($N \geq 79$ for circular conv. = linear conv.), we can conclude that we must zero-pad to the smallest power of 2 above 79, which is $N = 128$. We will need to pad 81 zeros to x and 95 zeros to h .

(c) Yes. This can be seen by noting that $x_{zp,e}[n]$ ($x[n]$ zero padded at the *end*) and $x_{zp,b}[n]$ ($x[n]$ zero padded on the *beginning*), as well as $h_{zp,e}[n]$ and $h_{zp,b}[n]$, can be related as follows:

$$\begin{aligned} x_{zp,b}[n] &= x_{zp,e}[\langle n - 81 \rangle_{128}], \\ h_{zp,b}[n] &= h_{zp,e}[\langle n - 95 \rangle_{128}]. \end{aligned}$$

Convince yourself of this by picturing $x_{zp,b}[n]$ and $x_{zp,e}[n]$; notice how if we circularly shift $x_{zp,e}[n]$ by a number equal to the number of padded zeros (81 in this case), the numbers slide to the right by 81 samples and the trailing 81 zeros come to the front. This forms exactly what we call $x_{zp,b}[n]$, leading to the above equivalence. By the circular shift property, the front-padded DFT's can be expressed

$$\begin{aligned} X_{zp,b}[k] &= e^{-j\frac{2\pi}{128}81} X_{zp,e}[k], \\ H_{zp,b}[k] &= e^{-j\frac{2\pi}{128}95} H_{zp,e}[k]. \end{aligned}$$

Notice that

$$X_{zp,b}[k]H_{zp,b}[k] = e^{-j\frac{2\pi}{128}81}X_{zp,e}[k]e^{-j\frac{2\pi}{128}95}H_{zp,b}[k] = e^{-j\frac{2\pi}{128}176}X_{zp,e}[k]H_{zp,e}[k] = e^{-j\frac{2\pi}{128}48}X_{zp,e}[k]H_{zp,e}[k].$$

If we define $z[n] = x[n] * h[n]$, which in (b) we showed is equal to $z[n] = x_{zp,e}[n] * h_{zp,e}[n]$, and define $\tilde{z}[n] = x_{zp,b}[n] * h_{zp,b}[n]$ as the output resulting in front-zero padding, we can use the circular shift property to show

$$\tilde{Z}_{zp,b}[k] = X_{zp,b}[k]H_{zp,b}[k] = e^{-j\frac{2\pi}{128}48}X_{zp,e}[k]H_{zp,e}[k] = e^{-j\frac{2\pi}{128}48}Z_{zp,e}[k] \implies \tilde{z}[n] = z[\langle n - 48 \rangle_{128}].$$

Thus, if we use front-padded sequences, the result will be the same as if we used back-padded sequences, only circularly shifted to the right by 48. To recover the correct sequence, thus, we must circularly shift this output to the left by 48.